



TECHNICAL REPORT

Segment Tree

Fast Range Queries and Dynamic Updates

Prepared by

SHAYAN UN NOOR
MD. SHADMAN SHAHRIYAR SHUVO
MD. WASIF HAQUE

Roll 2305024
Roll 2305025
Roll 2305029

Abstract

Segment Trees are hierarchical data structures designed to efficiently process range queries and dynamic updates over an array. A naïve approach may require linear time for every range query or update, which becomes inefficient when the number of operations is large. A Segment Tree addresses this problem by recursively partitioning an array into smaller intervals and storing a summary of each interval at a tree node. This organization allows both point updates and range queries to be performed in $O(\log n)$ time after $O(n)$ preprocessing.

This report presents the structure and construction of a Segment Tree, explains its update and query procedures using worked examples, derives their time complexities, and discusses its generalization to operations such as minimum, maximum, and greatest common divisor. It also introduces lazy propagation for range updates, provides a complete reference implementation, surveys representative applications, and compares Segment Trees with Fenwick Trees, sparse tables, and square-root decomposition.

Keywords: segment tree, range query, point update, associative operation, lazy propagation, Fenwick tree, logarithmic time.

Acknowledgements

The authors would like to thank the course instructors for their guidance throughout this project and for the opportunity to study a data structure that sits at the intersection of elegant theory and everyday engineering practice. The worked examples in this report were checked against independent reference computations, and the presentation slides that accompany it share the colour palette used in the figures below.

Contents

Abstract	ii
Acknowledgements	iii
List of Figures	v
List of Tables	vi
1 Introduction	1
1.1 Motivation	1
1.2 Dynamic Range Query Problem	2
1.3 Objective	2
1.4 Scope and Assumptions	2
1.5 Notation	3
1.6 Organization of the Report	3
2 The Range Query Problem	4
2.1 Problem Definition	4
2.2 Naïve Approach	4
2.3 Limitation of Prefix Precomputation	5
2.4 The Cost Trade-off Visualized	6
2.5 Need for a Dynamic Data Structure	6
3 The Segment Tree	7
3.1 Basic Idea	7
3.2 Structure of a Segment Tree	7
3.3 Interval Representation	8
3.4 Stored Information and Merge Operation	8
3.5 Structural Properties	8
3.6 Array Layout and Index Arithmetic	9
3.6.1 How Much Memory Is Enough?	9
3.7 Invariants	10

List of Figures

2.1	A single point update $A[4] \leftarrow 10$ invalidates five prefix sums (shaded red), $P[4]$ through $P[8]$. In general an update at index i invalidates $n - i + 1$ entries. . .	5
2.2	Cost per operation as a function of n . The linear cost of the two baselines eventually dominates, while the logarithmic Segment Tree cost is almost invisible on this scale.	6
3.1	Heap-style numbering of the Segment Tree for $n = 8$ (top) and the flat array that stores it (bottom). Node k has children $2k$ and $2k + 1$	9

List of Tables

1.1	Summary of notation.	3
2.1	Trade-off between query and update cost across approaches.	6
3.1	Level-by-level partition of $[1, 8]$ in the Segment Tree of the running example. .	7
3.2	Height and memory for several array sizes.	10

1

Introduction

Data rarely stays still. Prices tick, scores change, seats are booked and released, and sensors report new readings many times per second. Yet the questions that people ask of such data are almost never about a single value: they are about *ranges*. What is the total traded value between instruments 200 and 450? What was the lowest temperature recorded between 2 pm and 5 pm? Is any seat free in rows 10 through 14? Answering such questions quickly, while the underlying data continues to change, is the central problem studied in this report.

The Segment Tree is one of the most widely used answers to that problem. It is compact enough to be implemented in a few dozen lines, general enough to support a large family of aggregate operations, and efficient enough to serve millions of operations per second on modest hardware. This chapter motivates the problem, states the objective of the report, and outlines how the remaining chapters are organized.

1.1 Motivation

Many computational problems require repeatedly answering queries over a contiguous portion of a dynamically changing dataset. Consider a system tracking n financial instruments, where array A stores their current prices. At 9:31 AM, right after the market opens, 20,000 prices may change every second, while the system must still answer, instantly, “what is the total value of instruments l through r ?” The two fundamental operations required are therefore

$$\text{sum}(l, r) = \sum_{i=l}^r A[i] \tag{1.1}$$

and

$$\text{update}(i, v), \tag{1.2}$$

where $A[i]$ is modified to the value v .

The same pattern appears in very different settings. A multiplayer game must maintain a live leaderboard while thousands of players finish matches. A ticketing platform must find contiguous free seats while other customers are booking. A monitoring dashboard must report the maximum latency of the last five minutes while new measurements stream in. In each case the system is *summarizing a range and then changing part of it*, over and over.

1.2 Dynamic Range Query Problem

The difficulty arises from the interaction between queries and updates. If the array is scanned directly, a query in Equation (1.1) requires $O(n)$ time. If instead a running total is maintained, every update in Equation (1.2) requires $O(n)$ work to keep that total valid. Neither approach scales when both operations occur frequently and at large volume: keeping the sum fresh after every update costs $O(n)$ per update, while keeping updates instantaneous costs $O(n)$ per query. This tension — fast queries versus fast updates — appears in any large system that must *summarize a range and then change part of it*: stock feeds, leaderboards, and analytics dashboards all face it constantly.

Key Idea

Static data can be preprocessed once and queried forever. *Dynamic* data cannot: every preprocessing decision made to speed up queries must be paid for again each time the data changes. The Segment Tree is a principled compromise that keeps both costs logarithmic.

1.3 Objective

This report presents the Segment Tree as a data structure that resolves this tension, achieving $O(\log n)$ time for both operations after an $O(n)$ preprocessing step. We derive its structure, construction, update and query algorithms, and their time complexities; work through a concrete numerical example; generalize the structure to other associative operations; and compare it against alternative approaches. Concretely, the report aims to:

1. define the range-query problem precisely and expose why simple solutions fail (Chapter 2);
2. introduce the Segment Tree, its invariants, and its array layout (Chapter 3);
3. give and analyse algorithms for building, updating, and querying the tree (Chapters ??–??);
4. show how a single change of the merge operation yields range minimum, maximum, GCD and more (Chapter ??);
5. extend the structure to range updates through lazy propagation (Chapter ??);
6. provide a reference implementation (Chapter ??); and
7. place the structure among its alternatives and applications (Chapters ??–??).

1.4 Scope and Assumptions

Throughout the report we make the following assumptions, unless a section states otherwise.

- The array A has a fixed length $n \geq 1$ and is indexed from 1 to n .
- The merge operation is associative, has an identity element, and can be evaluated in $O(1)$ time.
- Updates replace a single value (*point update*). Range updates are treated separately in Chapter ??.
- Complexities are stated in the word-RAM model, counting one elementary arithmetic or comparison operation as one step.
- Timing figures given in Chapter ?? are order-of-magnitude illustrations, not benchmarks.

1.5 Notation

Table 1.1 collects the symbols used throughout the report.

Table 1.1. Summary of notation.

Symbol	Meaning
$A = [a_1, \dots, a_n]$	The underlying array of n elements (1-indexed).
$Q(l, r)$	Range query over the closed interval $[l, r]$.
$\text{update}(i, v)$	Point update: assign $A[i] \leftarrow v$.
$[lo, hi]$	The interval represented by a tree node.
mid	Midpoint $\lfloor (lo + hi)/2 \rfloor$ of a node's interval.
$S(l, r)$	Summary value stored for the interval $[l, r]$.
f, e	Associative merge operation and its identity element.
h	Height of the tree, $h = \lceil \log_2 n \rceil$.
q	Total number of operations (queries plus updates).
$P[i]$	Prefix sum $\sum_{j=1}^i A[j]$.

1.6 Organization of the Report

Chapter 2 formalizes the problem and studies the two baseline solutions. Chapter 3 introduces the Segment Tree and its structural properties. Chapters ??, ?? and ?? treat construction, point update, and range query in turn, each with a worked example on the running array

$$A = [2, 1, 5, 3, 7, 4, 6, 8].$$

Chapter ?? generalizes the merge operation, and Chapter ?? adds lazy propagation. Chapter ?? gives a complete implementation. Chapters ?? and ?? discuss applications and alternatives, Chapter ?? consolidates complexity results, and Chapter ?? concludes.

2

The Range Query Problem

Before introducing the Segment Tree it is worth understanding exactly why the problem is hard. This chapter defines the problem formally, analyses the two obvious solutions, and shows that they sit at opposite ends of a trade-off that neither can escape.

2.1 Problem Definition

Let

$$A = [a_1, a_2, \dots, a_n] \quad (2.1)$$

be an array of n elements. A range-sum query is defined as

$$Q(l, r) = \sum_{i=l}^r a_i, \quad 1 \leq l \leq r \leq n, \quad (2.2)$$

and a point update changes a single element,

$$a_i \leftarrow v. \quad (2.3)$$

A workload is a sequence of q operations, each of which is either a query or an update. The goal is to process the entire workload in as little total time as possible, and preferably to bound the cost of *each* operation individually.

Definition 1 (Dynamic range-query problem)

Given an array A of length n , maintain a data structure supporting (i) $Q(l, r)$, returning an aggregate of $A[l..r]$, and (ii) $\text{update}(i, v)$, setting $A[i] \leftarrow v$, such that the two operations may be interleaved in any order.

2.2 Naïve Approach

The most direct way to answer $Q(l, r)$ is to scan the range explicitly:

```
sum(l, r):  
    result = 0  
    for i = l to r:  
        result += A[i]  
    return result
```

For a range of length $r - l + 1$, this costs $O(r - l + 1)$ time, or $O(n)$ in the worst case:

$$T_{\text{query}}(n) = O(n). \quad (2.4)$$

A point update, by contrast, is trivially $O(1)$ under this scheme, since only $A[i]$ is written.

Example 2.1 (Cost of scanning). On $A = [2, 1, 5, 3, 7, 4, 6, 8]$, the query $Q(2, 7)$ reads six elements: $1 + 5 + 3 + 7 + 4 + 6 = 26$. On an array with $n = 10^6$ elements the same query may read up to a million elements, and if 10^5 such queries arrive per second the system must perform 10^{11} element reads per second, which is far beyond what a single core can sustain. $\triangleleft$

2.3 Limitation of Prefix Precomputation

An obvious optimization is to precompute prefix sums,

$$P[i] = \sum_{j=1}^i A[j], \quad (2.5)$$

so that any range sum can be answered in constant time via

$$Q(l, r) = P[r] - P[l - 1]. \quad (2.6)$$

For the running example the prefix array is

$$P = [2, 3, 8, 11, 18, 22, 28, 36],$$

and, for instance, $Q(3, 7) = P[7] - P[2] = 28 - 3 = 25$ on the original array.

This makes queries $O(1)$, but it introduces the opposite problem: whenever a single element $A[i]$ changes, every prefix sum $P[j]$ for $j \geq i$ becomes invalid, so an update costs $O(n)$ to restore correctness. Figure 2.1 shows the damage caused by one update.

	1	2	3	4	5	6	7	8
A before	2	1	5	3	7	4	6	8
P before	2	3	8	11	18	22	28	36
				update(4, 10)				
A after	2	1	5	10	7	4	6	8
P after	2	3	8	18	25	29	35	43

Figure 2.1. A single point update $A[4] \leftarrow 10$ invalidates five prefix sums (shaded red), $P[4]$ through $P[8]$. In general an update at index i invalidates $n - i + 1$ entries.

This gives the fundamental trade-off summarized in Table 2.1.

Table 2.1. Trade-off between query and update cost across approaches.

Method	Query	Point Update
Naïve array	$O(n)$	$O(1)$
Prefix sum	$O(1)$	$O(n)$
Segment Tree	$O(\log n)$	$O(\log n)$

2.4 The Cost Trade-off Visualized

The situation can be seen more clearly by plotting the cost per operation of each approach as n grows. In Figure 2.2 the naïve query cost and the prefix-sum update cost both grow linearly, while the Segment Tree cost for either operation stays close to the horizontal axis.

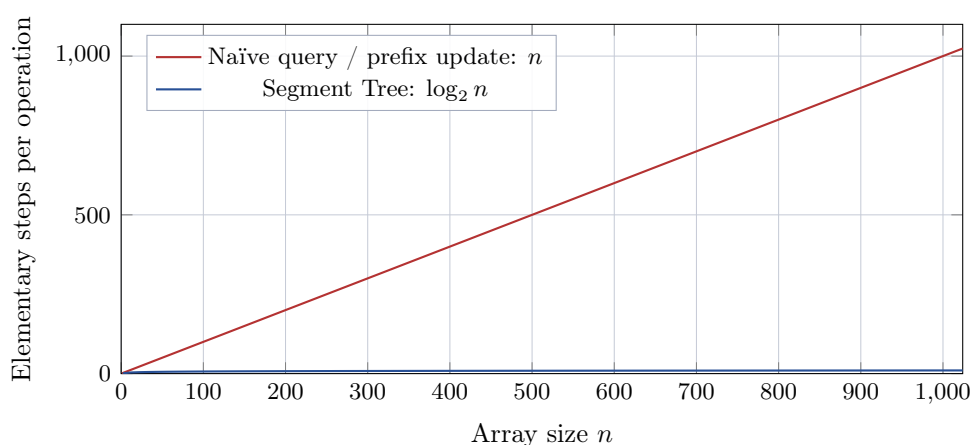


Figure 2.2. Cost per operation as a function of n . The linear cost of the two baselines eventually dominates, while the logarithmic Segment Tree cost is almost invisible on this scale.

2.5 Need for a Dynamic Data Structure

Neither extreme is acceptable when queries and updates are interleaved at scale. What is needed is a structure that balances both costs rather than optimizing one at the expense of the other. The Segment Tree achieves exactly this balance by decomposing the array into a hierarchy of intervals and storing a summary for each one, so that both operations touch only $O(\log n)$ nodes instead of $O(n)$ or $O(1)$ -with-a-hidden- $O(n)$ -cost elsewhere.

Remark

A useful way to think about the trade-off is as a question of *where* the work is done. The naïve array does all its work at query time; the prefix array does all its work at update time. The Segment Tree deliberately spreads the work: each update pays a small amount to keep a hierarchy of partial answers fresh, and each query pays a small amount to combine a handful of those partial answers.

3

The Segment Tree

This chapter introduces the Segment Tree as a data structure: what it stores, how its nodes correspond to intervals, and which structural properties make its operations efficient.

3.1 Basic Idea

A Segment Tree is a binary tree built over an array, where each node represents a contiguous sub-interval of the array and stores a summary value for that interval. The root represents the entire array; each internal node's interval is split into two halves, assigned to its left and right children; and each leaf represents a single array element. Instead of touching the whole array on every operation, the tree recursively splits the problem in half, solves each half, and combines the two answers — and only $O(\log n)$ splits are ever needed to reach any particular piece.

Key Idea

A Segment Tree precomputes the answer to a carefully chosen family of $\approx 2n$ intervals — the *dyadic* intervals obtained by repeated halving — such that every range $[l, r]$ can be written as a disjoint union of $O(\log n)$ of them.

3.2 Structure of a Segment Tree

For an array of size n , the tree has height $h = \lceil \log_2 n \rceil$, and consequently $O(n)$ nodes in total (at most $\sim 2n$ for a complete binary tree representation). Each level of the tree partitions $[1, n]$ into disjoint, contiguous sub-intervals that together cover the whole array exactly once.

Table 3.1 shows how the intervals at each level of the tree for $n = 8$ partition the array.

Table 3.1. Level-by-level partition of $[1, 8]$ in the Segment Tree of the running example.

Level	Intervals at this level	Nodes	Interval length
0	$[1, 8]$	1	8
1	$[1, 4], [5, 8]$	2	4
2	$[1, 2], [3, 4], [5, 6], [7, 8]$	4	2
3	$[1, 1], [2, 2], \dots, [8, 8]$	8	1
Total		15	

3.3 Interval Representation

Each node is identified by the interval $[lo, hi]$ it represents. If $lo = hi$, the node is a leaf corresponding to $A[lo]$. Otherwise, letting

$$mid = \left\lfloor \frac{lo + hi}{2} \right\rfloor, \quad (3.1)$$

the node's left child represents $[lo, mid]$ and its right child represents $[mid + 1, hi]$.

3.4 Stored Information and Merge Operation

For every node representing interval $[l, r]$, define the stored summary as

$$S(l, r) = \sum_{i=l}^r A[i]. \quad (3.2)$$

Given the midpoint $m = \lfloor (l + r)/2 \rfloor$, the summary satisfies the recursive relation

$$S(l, r) = S(l, m) + S(m + 1, r). \quad (3.3)$$

This is the *merge operation*: a parent's summary is obtained by combining its two children's summaries. For sum queries the merge is addition, but as shown in Chapter ??, any associative operation can be substituted in its place.

3.5 Structural Properties

Several simple facts about the tree are used repeatedly in later chapters. We state them here with short proofs.

Lemma 1 (Partition property)

For every level d of the tree, the intervals of the nodes at level d are pairwise disjoint, and their union is contained in $[1, n]$.

Proof. By induction on d . At level 0 there is a single node $[1, n]$. If the claim holds at level d , each node $[lo, hi]$ at that level is replaced at level $d + 1$ by the two disjoint intervals $[lo, mid]$ and $[mid + 1, hi]$ whose union is $[lo, hi]$; nodes with $lo = hi$ have no children. Disjointness and containment are therefore preserved. $\square$

Lemma 2 (Height)

A Segment Tree over $n \geq 1$ elements has height $\lceil \log_2 n \rceil$.

Proof. Each split at least halves the interval length, rounding the larger half up: an interval of length m splits into lengths $\lceil m/2 \rceil$ and $\lfloor m/2 \rfloor$. After d splits the longest interval therefore has length at most $\lceil n/2^d \rceil$, which equals 1 exactly when $2^d \geq n$, i.e. when $d \geq \lceil \log_2 n \rceil$. $\square$

Lemma 3 (Node count)

A Segment Tree over n elements has exactly $2n - 1$ nodes.

Proof. The tree is a full binary tree (every internal node has exactly two children) with n leaves. In any full binary tree the number of internal nodes is one less than the number of leaves, so the total is $n + (n - 1) = 2n - 1$. $\square$

For $n = 8$ this gives 15 nodes, matching Table 3.1.

3.6 Array Layout and Index Arithmetic

Segment Trees are almost always stored in a flat array rather than with explicit pointers. Numbering the root as node 1, the children of node k are

$$\text{left}(k) = 2k, \quad \text{right}(k) = 2k + 1, \quad \text{parent}(k) = \lfloor k/2 \rfloor. \quad (3.4)$$

This is the same layout used by binary heaps, and it removes the memory overhead of child pointers. Figure 3.1 shows the correspondence for the running example.

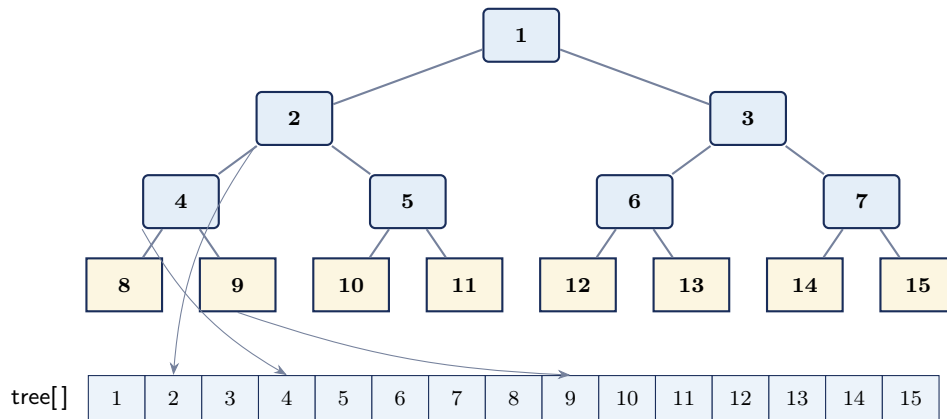


Figure 3.1. Heap-style numbering of the Segment Tree for $n = 8$ (top) and the flat array that stores it (bottom). Node k has children $2k$ and $2k + 1$.

3.6.1 How Much Memory Is Enough?

When n is not a power of two the tree is not perfect, and heap-style indices can reach values larger than $2n - 1$. A safe bound comes from Lemma 3.5: the deepest node has index below 2^{h+1} , where $h = \lceil \log_2 n \rceil$. Since $2^h < 2n$, we get $2^{h+1} < 4n$. This is why implementations conventionally allocate $4n$ cells, as in Table 3.2.

Remark

The true number of nodes is always $2n - 1$ (Lemma 3.5); the $4n$ allocation only pays for the unused gaps that appear in the heap-style numbering when n is not a power of two.

Table 3.2. Height and memory for several array sizes.

n	Height $\lceil \log_2 n \rceil$	Nodes $2n - 1$	Safe bound 2^{h+1}	Rule of thumb $4n$
8	3	15	16	32
100	7	199	256	400
1,000	10	1,999	2,048	4,000
1,024	10	2,047	2,048	4,096
10^6	20	1,999,999	2,097,152	4,000,000
10^9	30	1,999,999,999	2,147,483,648	4,000,000,000

3.7 Invariants

A correct Segment Tree maintains one invariant at all times, which every algorithm in the following chapters must preserve:

$$\text{for every node } v \text{ with interval } [l, r] : \quad \text{tree}[v] = \sum_{i=l}^r A[i]. \quad (3.5)$$

Construction establishes the invariant, queries only read it, and updates must repair it on exactly the nodes where it could have been broken.